

Cómo escribir tus specs

paso a paso

Detallado e ilustrado: locators, actions, data y los 3 tipos de escenario — por archivo y por preguntas.

1

Sesión 1 — Escenario básico

un step por acción

2

Sesión 2 — Agrupar acciones

varias acciones en un step (composite)

3

Sesión 3 — Scenario Outline

recorre toda la colección de datos

Qué es el spec y en qué orden se construye

Un "spec" es una receta en formato JSON que describe tu caso de prueba. El generador la lee y crea por ti todos los archivos del framework, ya listos para correr.

Qué archivos crea

Archivo	De qué parte del spec sale
src/pages/<Modulo>Page.ts	module, route, locators, actions
src/test/stepsDefinitions/<modulo>/...	scenarios[].steps
src/test/features/<modulo>/<Modulo>.feature	scenarios
jsonData/<env>/<modulo>.json	data
core/interfaces/<Modulo>Data.ts	campos de data (si hay datos)

El orden en que se arma un spec

- 1 module y route — lo PRIMERO. Definen el nombre del módulo y a qué pantalla apunta.
- 2 locators — los elementos de la pantalla (campos, botones, tablas).
- 3 actions — qué se hace sobre esos elementos (escribir, clic, verificar).
- 4 data — los datos de prueba (registros con id).
- 5 escenarios — los casos de prueba (los pasos y a qué acción se conecta cada uno).



INFO — Regla mental

Primero el "dónde" (module/route), luego el "qué hay" (locators), luego el "qué hago" (actions), luego "con qué datos" (data) y por último "el guion" (scenarios).

1

module y route (lo primero)

Es lo primero que defines. De aquí el generador deriva TODOS los nombres (clase, carpetas, archivos) y la navegación.

inicio del spec

```
{
  "module": "Login",
  "route": "/web/index.php/auth/login",
  "description": "Autenticación de usuarios",
  "envs": ["qa"]
}
```

- module — nombre del módulo. Se normaliza a PascalCase (ej. "Login" -> LoginPage, login/, Login.feature).
- route — la ruta de la pantalla, tras la URL base. Debe empezar con "/".
- description — opcional, va en la línea "Feature:".
- envs — opcional (default ["qa"]). Para qué ambientes crear el archivo de datos.

2 Locators — name, by, value

Un locator es la "dirección" de un elemento en la pantalla (un campo, un botón, una tabla). El generador los convierte en propiedades de tu página.

Por qué tres propiedades

- **name** — cómo lo llamas tú en el spec (camelCase). Con ese nombre lo referencian tus actions.
- **by** — la ESTRATEGIA para encontrarlo (cómo Playwright lo busca).
- **value** — el dato que usa esa estrategia (el texto del placeholder, el selector CSS, etc.).

locators

```
"locators": [  
  { "name": "usernameInput", "by": "placeholder", "value": "Username" },  
  { "name": "loginButton", "by": "role", "value": "button", "name_value": "Login" }  
]
```

Estrategias disponibles (by)

by	Se usa para	value de ejemplo
placeholder	Campo por su texto de ayuda	"Username"
role	Elemento por su rol (admite name_value)	"button" + name_value "Login"
label	Campo por su etiqueta	"Campaign Name"
text	Elemento por su texto visible	"Guardar"
testid	Atributo data-testid	"submit-btn"
css	Selector CSS directo	".oxd-table"
xpath	Expresión XPath	"//button[@type=submit]"



CONSEJO — role y name_value

Para "role", el value es el rol (button, link, textbox...) y name_value es el nombre accesible (el texto visible). Es la forma más robusta y recomendada por Playwright.

anchorLocator — el "ancla" de carga

Las páginas modernas cambian la URL antes de pintar el contenido. Si sigues de inmediato, capturas una pantalla en blanco o un elemento que aún no existe. El anchorLocator evita eso.

Qué es y qué pones

- Es el name de UN locator que confirma que la página ya cargó.
- `navigateTo()` navega a la route y ESPERA a que ese elemento sea visible antes de continuar.
- Elige algo estable que solo aparece cuando la página cargó: un botón principal, el título, una tabla.
- Es opcional: si no lo pones, se usa el PRIMER locator de tu lista.

```
anchorLocator
```

```
"anchorLocator": "loginButton"
```



ATENCIÓN — Qué NO usar de ancla

Evita elementos que desaparecen (un spinner de carga) o que tardan mucho. El ancla debe estar presente de forma estable cuando la pantalla terminó de cargar.

3

Actions — qué son y sus propiedades

Una "action" se convierte en un MÉTODO de tu página. Es lo que tu prueba "hace" sobre la pantalla. Cada acción se conecta a algo que el framework YA sabe hacer.

Propiedades de una action

Propiedad	Qué es	¿Cuándo?
name	Cómo se llama el método (camelCase)	siempre
type	Qué hace (del catálogo de abajo)	siempre
locator	Sobre qué elemento actúa (name de un locator)	acciones con elemento
label	Texto legible para la evidencia	opcional
expected	Texto esperado a verificar	assertText / assertAllText
fragment / pattern	Parte / patrón de la URL	assertUrl...
path / anchor	Ruta a navegar / ancla de espera	goto
uses	Lista de acciones que agrupa	composite

Catálogo de acciones del sistema

Estos son los "type" que puedes asociar a tus acciones. Cada uno mapea 1 a 1 a un método que el framework ya tiene (en BasePage / PageHelpers). Si usas un type que no está aquí, la validación FALLA — no se inventa nada.

type	Qué hace	Método del sistema	params
composite	Agrupar varias acciones en 1 step	(combina otras)	1 / 0
goto	Navega a una ruta (ej. login)	navigateAndCapture	0
fill	Escribe en un campo	fillField	1
click	Hace clic	clickElement	0
select	Elige opción de un <select>	selectOption	1
check	Marca/desmarca checkbox	checkElement	0
choose	Selecciona un registro/fila	chooseRecord	0
upload	Sube un archivo	uploadFile	1
assertVisible	Verifica que un elemento se ve	assertVisible	0
assertText	Verifica texto en un elemento	assertLocatorText	0
assertAllText	Verifica varios textos iguales	assertAllTextsEqual	0
assertUrlContains	La URL contiene algo	assertUrlContains	0
assertUrlMatches	La URL coincide con patrón	assertUrlMatches	0

"params" = cuántos valores entre comillas debe tener el step que use esa acción.

Cómo se asocia tu acción a la del sistema

spec

action: { name: "fillUsername", type: "fill", locator: "usernameInput" }



Page Object

genera el método `async fillUsername(value) { ... }`



Framework

que llama a `this.fillField(this.usernameInput, value)` (BasePage)

4

Data — cómo, por qué y dónde

Son los datos de prueba: una lista de registros (objetos JSON), cada uno con un id único. Separan los DATOS de la lógica de la prueba.

```
data
```

```
"data": [  
  { "id": "valido", "username": "Admin", "password": "admin123" }  
]
```

Por qué la necesito

- Para no quemar valores en el texto del escenario (mantenible y reutilizable).
- Para correr el mismo escenario con muchos datos (Scenario Outline).

Dónde se usa

- dataId — un step pasa UN campo del registro: getById(id).campo.
- composite — el método recibe el registro COMPLETO y usa varios campos.
- outline — el Examples se arma con los registros (una corrida por fila).



CONSEJO — Excluir registros con status

Agrega "status" a un registro para sacarlo de un Scenario Outline. Valores que excluyen: skip, inactive, disabled, off, false, no, 0, omit, omitir. Sin status, el registro se incluye.

Un escenario es un caso de prueba: una secuencia de pasos (steps). Cada step se VINCULA a una de tus acciones con "bind".

Propiedades de un step

Propiedad	Qué es
kw	Palabra Gherkin: Given / When / And / But / Then
text	La frase legible. Usa "comillas" para los valores variables.
bind	"navigate", el name de una action, o vacío (stub pendiente)
dataId	Opcional: el valor entre comillas es un id; pasa ese campo del registro

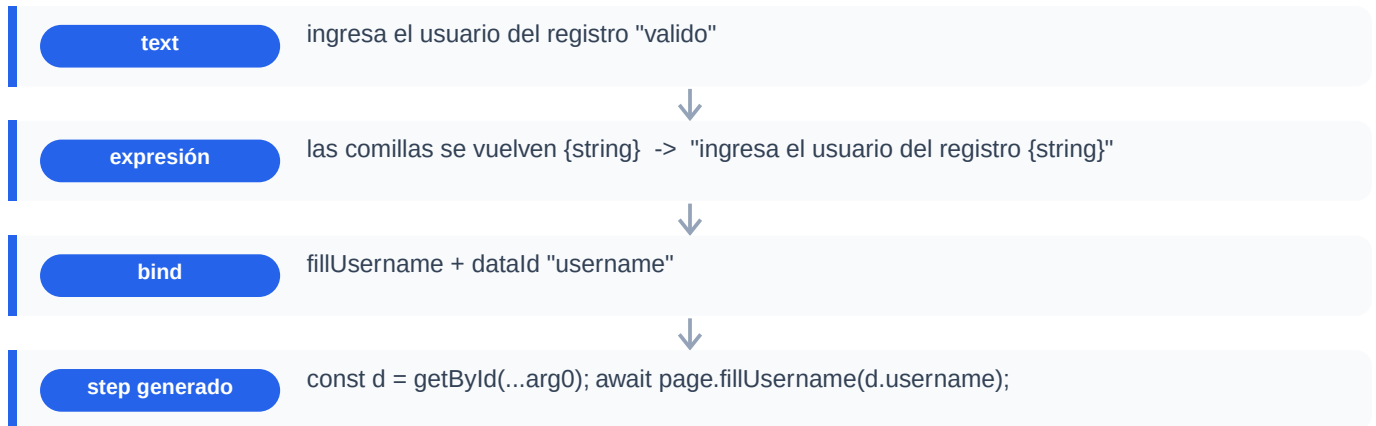
Los tres tipos de escenario

Tipo	Para qué	Marca en el spec
Básico	Un step por acción (detalle paso a paso)	scenario normal
Agrupar acciones	Varias acciones en 1 step (formularios)	una action "composite"
Scenario Outline	Recorre toda la colección de datos	"outline": true

Los tres se desarrollan completos en la Parte 2, con ejemplos reales.

Cómo se generan los steps

De cada frase ÚNICA del spec, el generador crea una definición de step. Tú solo describes la frase y a qué acción se conecta; el cuerpo lo arma el generador.



¿Qué info agrego sobre mis steps?

- kw y text — siempre.
- bind — a qué acción se conecta (o "navigate").
- dataId — solo si el valor sale de los datos.
- En Scenario Outline, usa el placeholder "<id>" en el text (Cucumber lo reemplaza por cada fila).



INFO — Reutilización entre módulos

Si una frase ya existe en otro módulo (ej. el login), el generador NO la vuelve a crear: la reutiliza. Por eso, para los pasos propios de tu módulo conviene usar frases únicas (incluye el nombre del módulo).

Los tags de Jira — NO los pongas tú

En el spec verás un campo "jira" en cada escenario. Déjalo SIEMPRE vacío ("").



IMPORTANTE — El sistema genera el tag, no tú

El tag @jira:KEY lo escribe AUTOMÁTICAMENTE el módulo de integración (QA Bridge) en el archivo .feature, una vez que el caso de prueba fue creado en Jira. Si lo pones a mano, interfieres con esa sincronización.

Cómo funciona (resumen)

- Corres las pruebas -> se ejecuta el dispatcher (Acción 1: Registro de Caso).
- Si el escenario no tiene caso en Jira, lo crea y escribe @jira:KEY en el .feature (FeatureTagger).
- En Scenario Outline, escribe un tag por fila automáticamente.

Detalle completo en docs/QA-BRIDGE-INTEGRATION-PROMPT.md (Acción 1 y patrón "Tagging automático del .feature").

El patrón más simple: cada acción es un step. Da el máximo detalle y una captura por paso. Ideal para empezar y para flujos cortos.

El spec completo

specs/examples/iniciar.spec.json

```
{
  "module": "Login",
  "route": "/web/index.php/auth/login",
  "description": "EJEMPLO BÁSICO – login paso a paso (acciones atómicas + datos)",
  "envs": ["qa"],

  "locators": [
    { "name": "usernameInput", "by": "placeholder", "value": "Username" },
    { "name": "passwordInput", "by": "placeholder", "value": "Password" },
    { "name": "loginButton", "by": "role", "value": "button", "name_value": "Login" },
    { "name": "sidebar", "by": "css", "value": ".oxd-main-menu" }
  ],
  "anchorLocator": "loginButton",

  "actions": [
    { "name": "fillUsername", "type": "fill", "locator": "usernameInput", "label": "campo Usuario" },
    { "name": "fillPassword", "type": "fill", "locator": "passwordInput", "label": "campo Contraseña" },
    { "name": "clickLogin", "type": "click", "locator": "loginButton", "label": "botón Login" },
    { "name": "assertDashboard", "type": "assertUrlContains", "locator": "sidebar", "fragment": "/dashboard/index", "label": "verifica redirección al dashboard" }
  ],

  "data": [
    { "id": "valido", "username": "Admin", "password": "admin123" }
  ],

  "scenarios": [
    {
      "title": "Login exitoso con credenciales válidas",
      "tags": ["Regresion", "Smoke"],
      "jira": "",
      "steps": [
        { "kw": "Given", "text": "el usuario está en la página de login", "bind": "navigate" },
        { "kw": "When", "text": "ingresa el usuario del registro \"valido\"", "bind": "fillUsername", "dataId": "username" },
        { "kw": "And", "text": "ingresa la contraseña del registro \"valido\"", "bind": "fillPassword", "dataId": "password" },
        { "kw": "And", "text": "hace clic en iniciar sesión", "bind": "clickLogin" },
        { "kw": "Then", "text": "el usuario es redirigido al dashboard", "bind": "assertDashboard" }
      ]
    }
  ]
}
```

Cómo se lee

- locators: los campos y el botón de la pantalla de login.
- actions: fillUsername, fillPassword, clickLogin (atómicas) y assertDashboard (verificación).
- data: un registro "valido" con username y password.

- escenario: 5 steps; cada When/And se conecta a una acción con bind; dataId saca el valor del registro.

El feature que genera

Login.feature (generado)

```
Scenario: Login exitoso con credenciales válidas
  Given el usuario está en la página de login
  When ingresa el usuario del registro "valido"
  And ingresa la contraseña del registro "valido"
  And hace clic en iniciar sesión
  Then el usuario es redirigido al dashboard
```

Sesión 1

Generar y luego correr

```
PS> npm run new:module -- specs/examples/iniciar.spec.json
```

```
PS> cross-env HEADLESS=true npm run test:qa
```

Para formularios grandes, un step por campo crea features enormes. Una acción "composite" agrupa varias acciones atómicas en UN método y UN step, alimentado por un registro de datos.

El spec completo

specs/examples/agrupar-acciones.spec.json

```
{
  "module": "LoginAgrupado",
  "route": "/web/index.php/auth/login",
  "description": "EJEMPLO COMPOSITE – agrupa varias acciones en UN solo step (usa 1 registro de datos)",
  "envs": ["qa"],

  "locators": [
    { "name": "usernameInput", "by": "placeholder", "value": "Username" },
    { "name": "passwordInput", "by": "placeholder", "value": "Password" },
    { "name": "loginButton", "by": "role", "value": "button", "name_value": "Login" },
    { "name": "sidebar", "by": "css", "value": ".oxd-main-menu" }
  ],
  "anchorLocator": "loginButton",

  "actions": [
    { "name": "fillUsername", "type": "fill", "locator": "usernameInput", "label": "campo Usuario" },
    { "name": "fillPassword", "type": "fill", "locator": "passwordInput", "label": "campo Contraseña" },
    { "name": "clickLogin", "type": "click", "locator": "loginButton", "label": "botón Login" },
    {
      "name": "iniciarSesion",
      "type": "composite",
      "label": "completa y envía el formulario de login",
      "uses": [
        { "action": "fillUsername", "field": "username" },
        { "action": "fillPassword", "field": "password" },
        { "action": "clickLogin" }
      ]
    },
    { "name": "assertDashboard", "type": "assertUrlContains", "locator": "sidebar", "fragment": "/dashboard/index", "label": "verifica redirección al dashboard" }
  ],

  "data": [
    { "id": "valido", "username": "Admin", "password": "admin123" }
  ],

  "scenarios": [
    {
      "title": "Login completo en un solo paso con datos válidos",
      "tags": ["Regresion", "Smoke"],
      "jira": "",
      "steps": [
        { "kw": "Given", "text": "el usuario abre el formulario de login", "bind": "navigate" },
        { "kw": "When", "text": "inicia sesión con el registro \"valido\"", "bind": "iniciarSesion" },
        { "kw": "Then", "text": "el usuario llega al dashboard", "bind": "assertDashboard" }
      ]
    }
  ]
}
```

```
}
```

Cómo se lee

- Defines las acciones atómicas (fillUsername, fillPassword, clickLogin) como siempre.
- Agregas una acción type "composite" con "uses": la lista de acciones que agrupa.
- En cada use, "field" indica de qué campo del registro sale el valor (para fill); las de clic no llevan field.
- El step pasa el ID del registro entre comillas; el composite usa sus campos. UN solo step hace todo el form.

Lo que genera

generado

```
// Page Object
async iniciarSesion(data: LoginAgrupadoData): Promise<void> {
  await this.fillUsername(data.username);
  await this.fillPassword(data.password);
  await this.clickLogin();
}

// Feature (un solo step para todo el formulario)
When inicia sesión con el registro "valido"
```



INFO — Las atómicas no necesitan step propio

fillUsername, fillPassword y clickLogin solo se usan dentro del composite, así que no las bindeas a ningún step. El feature queda mínimo.

Para correr el MISMO escenario una vez por cada registro de datos. El generador arma la tabla Examples a partir de tu data, y el campo "status" excluye registros.

El spec completo

specs/examples/scenario-outline.spec.json

```
{
  "module": "LoginNegativo",
  "route": "/web/index.php/auth/login",
  "description": "EJEMPLO OUTLINE – recorre toda la colección de datos; 'status' excluye registros",
  "envs": ["qa"],

  "locators": [
    { "name": "usernameInput", "by": "placeholder", "value": "Username" },
    { "name": "passwordInput", "by": "placeholder", "value": "Password" },
    { "name": "loginButton", "by": "role", "value": "button", "name_value": "Login" },
    { "name": "errorAlert", "by": "css", "value": ".oxd-alert-content-text" }
  ],
  "anchorLocator": "loginButton",

  "actions": [
    { "name": "fillUsername", "type": "fill", "locator": "usernameInput", "label": "campo Usuario" },
    { "name": "fillPassword", "type": "fill", "locator": "passwordInput", "label": "campo Contraseña" },
    { "name": "clickLogin", "type": "click", "locator": "loginButton", "label": "botón Login" },
    {
      "name": "iniciarSesion",
      "type": "composite",
      "label": "completa y envía el formulario de login",
      "uses": [
        { "action": "fillUsername", "field": "username" },
        { "action": "fillPassword", "field": "password" },
        { "action": "clickLogin" }
      ]
    },
    { "name": "assertError", "type": "assertText", "locator": "errorAlert", "expected": "Invalid credentials", "label": "verifica error de credenciales" }
  ],

  "data": [
    { "id": "pass-malo", "username": "Admin", "password": "ClaveMala1", "status": "active" },
    { "id": "usuario-malo", "username": "noexiste", "password": "admin123" },
    { "id": "vacios", "username": "", "password": "", "status": "skip" }
  ],

  "scenarios": [
    {
      "title": "Login inválido por cada caso negativo",
      "tags": ["Regresion"],
      "jira": "",
      "outline": true,
      "examplesColumn": "id",
      "steps": [
        { "kw": "Given", "text": "el usuario abre la pantalla de autenticación", "bind": "navigate" },
        { "kw": "When", "text": "intenta iniciar sesión con el registro \<id>", "bind": "iniciarSesion" }
      ]
    }
  ]
}
```



```
        { "kw": "Then", "text": "se muestra el error de credenciales inválidas", "bind":  
"assertError" }  
      ]  
    }  
  ]  
}
```

Cómo se lee

- "outline": true marca el escenario como Scenario Outline.
- "examplesColumn" (default "id") indica qué campo alimenta la tabla Examples.
- En el text usas el placeholder "<id>" — Cucumber lo reemplaza por el valor de cada fila.
- El registro con "status": "skip" queda excluido: no aparece en Examples ni se ejecuta.

El feature que genera

LoginNegativo.feature (generado)

Scenario Outline: Login inválido por cada caso negativo

Given el usuario abre la pantalla de autenticación
When intenta iniciar sesión con el registro "<id>"
Then se muestra el error de credenciales inválidas

Examples:

id
pass-malo
usuario-malo



LISTO — vacíos quedó fuera

El registro "vacíos" tenía status "skip", por eso no aparece en la tabla Examples. El Outline corre solo pass-malo y usuario-malo.

La sesión de preguntas (wizard)

Si prefieres no escribir el JSON, ejecuta el comando SIN argumentos y el asistente te hace preguntas. Al final guarda el mismo spec y genera todo. Sirve para los tres tipos de escenario.

Iniciar el wizard

```
PS> npm run new:module
```

- Una respuesta vacía (solo Enter) termina la lista actual (locators, acciones, datos, steps...).
- Al final pregunta si guardar el spec en specs/<modulo>.spec.json.
- NO pregunta por el tag de Jira: ese lo genera el sistema después (queda vacío).

En azul la pregunta; en verde lo que TÚ escribes.

Wizard — Sesión 1 (escenario básico)

wizard — básico

```
Nombre del módulo: Login
Route: /web/index.php/auth/login
Descripción (opcional): Autenticación
Envs [qa]: qa
Locators (Enter en name para terminar)
  locator name: usernameInput
  by: placeholder
  value: Username
  ... (passwordInput, loginButton, sidebar igual) ...
  locator name: (Enter)
anchorLocator [usernameInput]: loginButton
Acciones (Enter en name para terminar)
  action name: fillUsername
  type: fill
  locator: usernameInput
  label: campo Usuario
  ... (fillPassword, clickLogin, assertDashboard) ...
Datos (Enter en id para terminar)
  data id: valido
  campos: username=Admin,password=admin123
Escenarios
  Scenario title: Login exitoso
  tags: Regression
  ¿Scenario Outline? [s/N]: N
    kw: When
    text: ingresa el usuario del registro "valido"
    bind: fillUsername
    dataId: username
    ... (resto de steps) ...
  ¿Guardar el spec? [S/n]: S
```

Wizard — Sesiones 2 y 3 (composite y outline)

El wizard también arma composite y outline. Aquí las partes que cambian respecto al básico.

Sesión 2 — composite (en el bloque de Acciones)

Primero creas las acciones atómicas; luego una acción de type "composite", y el wizard te pide sus sub-acciones (uses):

wizard – composite

```
... ya creaste fillUsername, fillPassword, clickLogin ...
action name: iniciarSesion
type: composite
  Sub-acciones del composite (Enter en action para terminar)
  use action: fillUsername
  field: username
  use action: fillPassword
  field: password
  use action: clickLogin
  field: (Enter, no recibe valor)
  value: (Enter)
  use action: (Enter termina)
label: envía el formulario
Luego, en el escenario, un solo step:
  text: inicia sesión con el registro "valido"
  bind: iniciarSesion
```

Sesión 3 — outline (en el bloque de Escenarios)

Al crear el escenario, responde "s" a la pregunta de Outline e indica la columna. Marca un registro con status=skip para excluirlo:

wizard – outline

```
En Datos, un registro excluido:
  data id: vacíos
  campos: username=,password=,status=skip
En el escenario:
  Scenario title: Login inválido por caso
  tags: Regresion
  ¿Scenario Outline? [s/N]: s
  examplesColumn [id]: id
  usa el placeholder "<id>" en el text:
    text: intenta iniciar sesión con el registro "<id>"
    bind: iniciarSesion
```



LISTO — Mismo resultado por ambos caminos

El wizard guarda specs/<modulo>.spec.json idéntico al que escribirías a mano, y genera los mismos archivos. Después solo verificas los selectores reales y corres las pruebas.